```
  test "a project with an incomplete task is not done" do
    add_one_task
    refute(@project.done?)
  end

  test "a project with complete tasks is done" do
    add_one_task
    @task.mark_completed
    @task.save
    assert(@project.done?)
  end

  test "can calculate sizes" do
    add_many_tasks
    assert_equal(10, @project.size)
    assert_equal(6, @project.incomplete_tasks.map(&:size).sum)
    assert_equal(3, @project.completed_velocity)
    assert_equal(1.0 / 7, @project.current_rate)
    assert_equal(42, @project.projected_days_remaining)
  end
end
```

This is the same set of tests, with a couple of minor changes—most notably, the Minitest version saves test data to the database so that it can use full Rails ActiveRecord associations.

Instead of three different `describe` blocks, the Minitest version has three different setup methods—the basic `setup` that creates a project, and the `add_one_task` and `add_many_tasks` methods. The two methods are called at the beginning of whatever tests need them. The Minitest version has the advantage of being quite explicit about what setup happens in what test. RSpec is often criticized for being opaque here, especially if the `describe` blocks are nested. The cost is a little bit more typing and a little bit less structure. In the Minitest version there's no grouping of tests, so it can be harder to determine which tests are related.

To clear up one other difference between this Minitest example and the RSpec example, this Minitest example uses factory_bot's `create` build strategy rather than `build_stubbed`. This is kind of incidental to Minitest—you could write a Minitest version that used `build_stubbed`; it's just that the limitations of `build_stubbed` not being able to touch the database conflicted with the setup structure for creating the associations, so I decided it was clearer to create the database items. I did combine all the sizing assertions into one test to minimize the amount of setup.